# IoT Application

2110575 · Part App · Week 1–3 · Quiz 14 ต.ค. 09:00–10:00



## กติกาสอบ & คะแนน

| | |
|---|---|
| เวลา | 1 ชม. · 09:00–10:00 · 14 ต.ค. |
| รูปแบบ | MCQ ทั้งหมด (ตัวเลือก A–E) |
| พกได้ | **A4 หน้า–หลัง 1 แผ่น เท่านั้น** |
| ขอบเขต | Week 1–3 (Lab 1–3) · ไม่รวม Lab 4 |
| หลังสอบ | 10:00–12:00 Lab 4 Image Captioning |

คะแนน: Lab 1–4 ละ 4% = 16% (กลุ่ม 3 / เดี่ยว 1) · Quiz 10% → รวม 25% + bonus 1% (เพดาน 25%) · ข้อสอบเก่า 8 ต.ค. 2025 = **32 ข้อ ข้อละ 1 คะแนน**

## IoT คืออะไร · 9 Characteristics

- **IoT / IoE** = เครือข่ายของ things + platform ที่คุยกันเองผ่าน wireless โดยไม่ต้องมีมนุษย์สั่ง *(without human intervention)* → ได้ efficiency, performance, safety
- **3 ประโยชน์หลัก:** Communication · Control & Automation · Cost Saving
- **9 คุณลักษณะ (ท่องให้ได้):** Connectivity · Intelligence · Dynamic Nature · Enormous Scale · Sensing · Heterogeneity · Security · Unique Identity · Integration into Information Network

**กับดักข้อสอบจริง:** ถามว่าอะไร **ไม่ใช่** คุณลักษณะของ IoT → ตอบ **High latency** (latency สูงเป็นข้อเสีย ไม่ใช่คุณลักษณะ) · ระวัง **Scalability** ที่เป็นคำตอบลวง เพราะมันคือ **Enormous Scale** จริง ๆ

## ภาพรวม 4 ชั้นของระบบ IoT

- **1. Devices & Sensors** — เก็บข้อมูลจากโลกจริง (อุณหภูมิ การเคลื่อนไหว สุขภาพ) · MCU/edge: Arduino, ESP32, Raspberry Pi
- **2. Communication & Networking** — Wi-Fi, BLE, ZigBee, LoRa, 5G · โปรโตคอล IPv6, 6LoWPAN, MQTT, CoAP
- **3. Data Processing & Platforms** — edge หรือ cloud · storage, filtering, analytics, AI/ML → เปลี่ยน raw data เป็น information
- **4. Applications** — Smart Home · Smart City · Healthcare · Transportation

**Edge device ที่นับ: Raspberry Pi, ESP32, Arduino, Jetson Nano — Desktop PC ไม่นับเป็น edge device**

## Use Cases — Smart City 2 เคส

| | SINGAPORE | BARCELONA |
|---|---|---|
| จุดเน้น | citizen-centric บริการประชาชนทั่วประเทศ | urban infrastructure & sustainability |
| ชื่อโครงการ | Smart Nation | Smart City |
| แพลตฟอร์ม | รัฐบาลผลักดัน ใช้ในชีวิตประจำวัน | Sentilo / CityOS (open data) |
| ตัวอย่าง | MRT/bus tracking · e-payment ไร้เงินสด · telemedicine + wearables · automated immigration (ใบหน้า+ม่านตา) · smart water meter · environmental modelling · robotics & drones | smart parking · smart lighting ปรับความสว่างเอง · smart waste (ถังมีเซนเซอร์ จัดเส้นทางเก็บ) · air quality · smart irrigation |

## Communication Models ออกสอบแน่

| MODEL | ลักษณะ · โปรโตคอล |
|---|---|
| **D2D** Device-to-Device | อุปกรณ์คุยกันตรง ๆ ไม่ผ่าน cloud/gateway · **MQTT · CoAP · WebSocket · BLE** · เช่น smartwatch ส่ง step count เข้ามือถือทาง Bluetooth |
| **D2C** Device-to-Cloud | อุปกรณ์ยิงขึ้น cloud ตรง ๆ ไม่ผ่าน gateway · **REST API (HTTP/HTTPS) · MQTT · gRPC** · เช่น fitness tracker อัป heart rate/sleep ขึ้น cloud เอง · thermostat → AWS IoT Core |
| **D2G** Device-to-Gateway | เซนเซอร์ → gateway (Pi / router / มือถือ) รวบรวมก่อนส่งต่อ · **MQTT · Zigbee · BLE** · เช่น เซนเซอร์ smart home/farm ส่งเข้า Pi |
| **Data sharing** / Back-end (C2C) | Cloud-to-Cloud แลกข้อมูลกันข้ามแพลตฟอร์ม · Fitbit Cloud → Google Fit → ระบบโรงพยาบาล · Azure IoT Hub → SAP → supplier |

**ตัวตัดสิน:** มี gateway ตรงกลางไหม → ไม่มี + ขึ้น cloud = D2C · ไม่มี + ไม่ขึ้น cloud = D2D · มี = D2G · cloud คุย cloud = C2C

## Data Storage & Database ใน IoT

- **1 · แบ่งตามที่เก็บ:** On-Device — SD card, embedded DB · Edge — gateway / edge server, latency ต่ำ · Cloud — scale ได้ ต่อ big data & ML
- **2 · แบ่งตาม DATA MODEL:** Relational (SQL) — MySQL, PostgreSQL · NoSQL — Key-Value: Redis, DynamoDB · Document: MongoDB, CouchDB · Time-Series: InfluxDB, TimescaleDB · Graph: Neo4j · Object Storage (รูป/วิดีโอ) · Vector DB
- **3 · แบ่งตาม USE CASE:** Streaming — Kafka, RabbitMQ, MQTT broker · Analytical / Data Lake — BigQuery, Redshift, Snowflake · IoT-specific — AWS Timestream, Azure Cosmos DB, Google Firebase

ข้อมูล sensor ตามเวลา → **Time-Series DB** · รูป/วิดีโอ → **Object Storage**

## Processing Models

| MODEL | ข้อดี | ข้อเสีย | ใช้ที่ไหน |
|---|---|---|---|
| **Cloud-centric** ส่ง raw ขึ้น cloud แล้วส่งผลกลับ | พลังประมวลผลสูง · รองรับ big data · อัปเดตโมเดลง่าย · เทรน ML ได้ | latency สูง · ต้องมีเน็ตตลอด · เสี่ยง privacy | Smart City, Smart Agriculture, healthcare analytics |
| **Edge-only** ประมวลผลบนตัวอุปกรณ์ | latency ต่ำสุด · ทำงาน offline ได้ · privacy ดี | ทรัพยากรจำกัด · โมเดลเล็กเท่านั้น · อัปเดตยาก | รถไร้คนขับ, wearable, CCTV real-time |
| **Hybrid** Edge + Cloud | สมดุล latency · ลด bandwidth · ได้ทั้งสองโลก | สถาปัตยกรรมซับซ้อน · แพงกว่า · ต้องทำ security สองฝั่ง | Smart Factory, predictive maintenance, Smart Healthcare |

**สูตรเลือกข้อ:** โจทย์มีทั้ง real-time ที่ปลายทาง และวิเคราะห์ ML ข้อมูลก้อนใหญ่ → ตอบ **Hybrid** ทุกครั้ง

## EDGE VS FOG VS CLOUD

| | EDGE | FOG | CLOUD |
|---|---|---|---|
| ที่ประมวลผล | บนอุปกรณ์/เซนเซอร์/gateway | ชั้นกลาง local server | data center รวมศูนย์ |
| Latency | ต่ำสุด real-time | ต่ำ–กลาง | สูงกว่า |
| การเชื่อมต่อ | offline ได้ | ต้องมี local network | ต้องมีอินเทอร์เน็ต |

## IoT Levels 4 ระดับ สับสนบ่อย

| ระดับ | ทำอะไร | ตัวอย่าง |
|---|---|---|
| 1 Measurement | เก็บ raw data จากสิ่งแวดล้อมด้วยเซนเซอร์ | เซนเซอร์ความชื้นดินตรวจว่าดินแห้ง |
| 2 Automation | สั่งทำงานอัตโนมัติจากค่าที่วัดได้ (if-else / threshold) | ปั๊มน้ำเปิดเอง · ประตูล็อกเปิดเองเมื่อเจอคนที่มีสิทธิ์ |
| 3 Innovation | สร้างบริการ/โมเดลธุรกิจใหม่จากข้อมูล IoT + AI/analytics | ผูกทะเบียนกับแอป ตัดเงินอัตโนมัติ ปรับราคาตามช่วงเวลา · บริการ smart farming ที่ติดตามประสิทธิภาพ |
| 4 Cloud Storage | แพลตฟอร์มกลางสำหรับเก็บ แชร์ วิเคราะห์ | เก็บข้อมูลไว้วิเคราะห์ผลผลิต |

**Automation vs Innovation:** อ่านป้ายทะเบียน → ถ้าอยู่ในลิสต์ไม้กั้นเปิดเอง = **Automation** · เพิ่มจ่ายเงินผ่านแอป + วิเคราะห์ช่วงคนแน่น + ปรับราคาตามเวลา = **Innovation**

## User-facing IoT Apps — 3 หน้าที่

- 1 **Visualization** — เห็นค่าสด ๆ แบบ real-time เพราะค่า sensor เปลี่ยนตลอด · dashboard smart parking อัปเดตทันทีที่รถเข้า/ออก ไม่ใช่รอ 5 นาที · เกม: โชว์ท่ามือที่ AI ตรวจได้ ("ROCK") บนจอทันที
- 2 **Notification & alerts** — ระบบ push ให้ผู้ใช้เอง ไม่ต้องรอผู้ใช้มาเปิดดู (threshold / anomaly) · wearable แจ้งเตือนเมื่อ heart rate > 120 bpm · เกม: ขึ้นผล "You Win!" ทันที = Notification
- 3 **Control loop** — ผู้ใช้สั่งแล้วอุปกรณ์เปลี่ยนสถานะจริง — IoT คือ interaction ไม่ใช่แค่ monitoring · กด "Turn on AC" แล้วคำสั่งวิ่งถึงเครื่องทันที · เกม: ปุ่มเริ่มรอบใหม่

**Performance:** ถ้า latency > 1 วินาที เกมเป่ายิ้งฉุบเล่นไม่ได้ · ถ้าใช้ 2 กล้อง (เล่นข้ามเครื่องแบบ LINE) ต้องเพิ่ม sync สองฝั่ง + ส่ง state ข้ามเครือข่าย → กลายเป็น D2C/C2C

## โปรโตคอลที่โผล่ในวิชานี้

| PROTOCOL | รูปแบบ | ใช้เมื่อ |
|---|---|---|
| **MQTT** | Publish/Subscribe ผ่าน broker · TCP | เซนเซอร์จำนวนมาก แบนด์วิดท์ต่ำ · D2D/D2G/D2C |
| **CoAP** | REST-like บน UDP เบามาก | อุปกรณ์จิ๋ว พลังงานต่ำ · D2D |
| **HTTP/HTTPS** | REST API Request/Response | คุย cloud/API · สตรีมวิดีโอ MJPEG ผ่าน Flask ในแล็บ 1 & 3 |
| **WebSocket** | สองทางเต็ม คาไว้ตลอด | dashboard real-time |
| **gRPC** | RPC บน HTTP/2 · binary | service คุยกันในระบบใหญ่ |
| **ZigBee / BLE / LoRa** | วิทยุระยะสั้น–ไกล พลังงานต่ำ | D2D / D2G ใน smart home, farm |

**6LoWPAN** = IPv6 บนเครือข่ายไร้สายพลังงานต่ำ

# Evaluation · Lab 1–3

Camera & Haar · OCR · MediaPipe · เมตริกและกับดัก



## IoT Evaluation 7 หัวข้อ ออกหลายข้อ

| หัวข้อ | ความหมาย · ตัวอย่าง |
|---|---|
| **1 Measurement Accuracy** | เซนเซอร์/AI วัดถูกไหม · soil ±2% · detection ≥ 85% · วิธีตรวจ: เทียบกับเครื่องมือ ground-truth ที่ calibrate แล้ว ตาม datasheet |
| **2 Automation & Responsiveness** | สั่งทำงานเองจากค่าที่วัด + ตอบสนองไว · ไฟติดใน 0.5 วิ · เตือน HR > 120 bpm ถูก 90% |
| **3 Innovation & Usefulness** | สร้างคุณค่าใหม่เกินการวัด/ควบคุมธรรมดา · ถังขยะ AI แยกขยะ · ห้องเรียนวัดความสนใจจากสีหน้า · healthcare → บริการแพทย์แบบใหม่ที่เมื่อก่อนทำไม่ได้ |
| **4 Cloud Integration & Data Handling** | ขึ้น cloud เพื่อเก็บ/แสดง/วิเคราะห์ · PM2.5 → Firebase → แอป · Node-RED + MQTT → กราฟสด · excellent = connectivity ≥ 95% เก็บยาวไม่สูญหาย |
| **5 System Performance** | efficiency + reliability + scalability · uptime 95% 7 วัน · 4 กล้อง latency < 1 วิ · แบต 12 ชม. |
| **6 User Experience & Presentation** | ใช้ง่าย สื่อสารชัด · Chart.js / Node-RED · ปุ่ม ON/OFF · ไดอะแกรม + live demo · เมตริกที่ถูก: **เวลาเฉลี่ยที่ผู้ใช้เรียนรู้การใช้แอป** (ไม่ใช่ CPU / API req / โควตา) |
| **7 Cost Efficiency & Feasibility** | Hardware · Cloud free-tier vs paid · CAPEX/OPEX · Cost–Benefit · Sustainability · farming < 2,000฿ (ESP32 300 + soil 150 + กล้อง 1,000) · parking ~8,000฿ (กล้อง 2,000 + Jetson 4,000 + เน็ตเตล็ด 2,000) + AWS จ่ายต่อ request |

## AI-Specific Evaluation 6 หัวข้อ

- **1. Model Accuracy** — Precision / Recall / F1
- **2. Inference Latency** — edge vs cloud
- **3. Resource Efficiency** — Memory, CPU/GPU → ข้อ **"แบตต้องอยู่ 12 ชม."** ตอบข้อนี้
- **4. Robustness & Fairness** — ทนทุกสภาพ ไม่มีอคติ
- **5. Explainability (XAI)** — อธิบายเหตุผลได้ → **หมอ/ผู้ใช้เชื่อถือผล AI**
- **6. Maintainability** — retrain/อัปเดต MLOps → **โมเดลปี 2023 เจอโรคพืชใหม่ปี 2025 แล้วแย่ = ข้อนี้**

## Lab 1 · Camera + Face Detection + Blur

ไปป์ไลน์: Frame capture (OpenCV) → Face Detection (Haar Cascade) → Privacy Blur → Logging → Flask Web Stream

- สายกล้อง Pi = **MIPI CSI-2 serial** (ไม่ใช่ USB/UVC, I²C, SPI, HDMI)
- สตรีมขึ้นเบราว์เซอร์ = **Flask + HTTP** (MJPEG) ไม่ใช่ MQTT/WebSocket/gRPC
- เช็ค OS `cat /etc/os-release` · เปิดกล้องรุ่นเก่า `raspi-config → Interfacing Options → Camera`
- คำสั่งกล้อง: Bullseye `libcamera-hello / -jpeg -o x.jpg / -vid -o x.h264 / -raw` → Bookworm เปลี่ยนคำนำหน้าเป็น `rpicam-*`

### HAAR CASCADE

- Sliding Window + Haar Filters ไถทั่วภาพ
- Integral Image → หาผลรวมพื้นที่เร็ว (fast sum)
- Cascade Stages → ด่านแรกตัดพื้นหลังทิ้งไว ๆ

```
# ต้องแปลง grayscale ก่อนเสมอ
f = cv2.CascadeClassifier(cv2.data.haarcascades +
    'haarcascade_frontalface_default.xml')
faces = f.detectMultiScale(gray,
  scaleFactor=1.1,  # ↑ เร็วขึ้น แม่นลง
  minNeighbors=5,   # ↑ false positive ลด
  minSize=(30,30))
for (x,y,w,h) in faces:
  roi = frame[y:y+h, x:x+w]
  frame[y:y+h, x:x+w] = cv2.blur(roi,(25,25))
```

**fps:** processing time/เฟรม = t_end − t_start · **fps = 1 / processing time (s)** → 50 ms = 20 fps · ในแล็บค่า default ช่วงที่หน้าเริ่มเบลอ = **11–25 fps**

## Spatial Filtering vs Convolution

- **Filtering** = ไถ kernel ทับภาพ หาผลรวมของผลคูณ → ได้ correlation ของภาพกับ kernel
- **Convolution** = เหมือนกัน แต่ **พลิก (mirror) kernel 180°** ก่อน → output คล้าย kernel · kernel สมมาตร = ผลเท่ากัน · ใช้ `cv2.filter2D`
- **Box/Average (cv2.blur)** = เบลอ · kernel ใหญ่ขึ้น 5×5 → 15×15 = **เบลอแรงขึ้น + ช้าลง** (ไม่ใช่คมขึ้น ไม่ใช่ fps ขึ้น)
- **Gaussian** = เบลอนุ่ม ถ่วงน้ำหนักกลางมาก
- **Laplacian** = เหลือแต่ขอบ ไม่เหลือรายละเอียดอื่น (อนุพันธ์อันดับ 2) ขยาย noise
- **Sobel/Prewitt** = ขอบตามแกน x หรือ y · **Sharpen** = คมขึ้น

## Lab 2 · OCR อ่านป้ายทะเบียน

เปลี่ยน raw image → ข้อมูลที่ใช้ได้ · ไลบรารี = **EasyOCR** (ไม่ใช่ OpenCV/NumPy/Matplotlib/Pandas) · โชว์ผลบนเว็บ Flask/Gradio · ใช้กับ smart logistics, smart parking, document digitization

### 4 กลุ่ม OCR

- **Basic** Tesseract 5–20 MB · EasyOCR 30–80 MB → ฟรี เบา CPU ได้ แต่แม่นน้อย ฟอนต์ไทยแย่ ไม่วิเคราะห์ layout · รันบน Pi/edge
- **Cloud** Google Vision, Azure OCR, AWS Textract → แม่นหลายภาษา ต่องาย แต่ต้องมีเน็ต + มีค่าใช้จ่าย + เสี่ยง privacy
- **Advanced** PaddleOCR mobile 8–20 MB / server 100–300 MB · MMOCR 50–200 MB · DocTR 200–400 MB · Keras-OCR 70–120 MB → detection+recognition, layout ซับซ้อน แต่หนัก ต้อง GPU
- **Native/Mobile** iOS Vision, Android ML Kit → on-device offline แต่ผูกแพลตฟอร์ม

### พารามิเตอร์ EASYOCR

- **text_threshold** เกณฑ์ความมั่นใจของตัวอักษร · เจอขยะมั่ว ๆ ทั่วภาพ → เพิ่มค่านี้
- **low_text** ขอบล่างคะแนนข้อความ · ลด = จับตัวจาง ๆ ได้มากขึ้น (ขยะมากขึ้น)
- **link_threshold** เชื่อมตัวอักษรเป็นคำ · **gpu=False** บังคับ CPU
- **allowlist** จำกัดชุดอักขระ (ไทย+เลข) → ลดการอ่านผิดประเภท ไม่ได้ช่วยความเร็วเซิร์ฟเวอร์ / contrast / การหมุน

```
r = easyocr.Reader(['th','en'], gpu=False)
out = r.readtext(img, allowlist=ALLOWLIST,
       text_threshold=0.7, low_text=0.4)
# out = [(bbox, text, confidence), ...]
```

**เร่งความเร็ว:** crop ROI ป้าย / ย่อภาพ / grayscale · ทำ OCR ทุก N เฟรม · โหลด Reader ครั้งเดียวนอกลูป · ใช้ allowlist + โมเดลเบา · ยกงานหนักขึ้น cloud (hybrid)

### เมตริกที่ต้องเลือกให้ถูก

- **Accuracy** = % ที่ถูกทั้งหมด (exact match) → ไม้กั้นเปิดเฉพาะเมื่ออ่านป้ายถูกทุกตัว ผิดตัวเดียวไม่ผ่าน = Accuracy
- **CER** = (S+D+I)/N ตัวอักษร · CER 15% = ตัวอักษร 15% ไม่ตรง (ไม่ใช่ถูก 85% ไม่ใช่เฟรมหลุดไม่ใช่ fps/latency) · **WER** = ระดับคำ
- **Precision** TP/(TP+FP) ที่ทายว่าใช่ ถูกจริงกี่ % · **Recall** TP/(TP+FN) ของจริงจับได้กี่ % · **F1** = 2PR/(P+R) ใช้เมื่อคลาสไม่สมดุล
- **BLEU** = งานแปล/generate text · **OKS** = งาน keypoint/pose — ทั้งคู่ไม่ใช่เมตริก OCR ป้ายทะเบียน
- ความเชื่อถือได้ของระบบเตือน = % ของ alert ที่ส่งถึงสำเร็จภายใน 5 วินาที (ไม่ใช่จำนวนผู้ใช้ ไม่ใช่รอบชาร์จแบต)
- **Latency / fps** = ความเร็ว · **uptime %** = reliability

## Lab 3 · MediaPipe Hands + เป้ายิ้งฉุบ

ไปป์ไลน์: Picamera2 init (resolution + format) → capture + แปลงรูปแบบให้ MediaPipe (BGR→RGB) → Hand Landmark Detection → เก็บ y ของ TIP/PIP → classifier() → วาด landmark + ข้อความท่า → encode MJPEG → Flask Response → render HTML template

- **MediaPipe Hands = 21 landmarks ต่อมือ** (ไม่ใช่ 19/20/22/23) · index 0 = WRIST · lm.x, lm.y เป็น 0–1 ต้องคูณ frame width/height
- ข้อนิ้ว โคน → ปลาย: **MCP** (โคนมือ) → **PIP** (ข้อกลาง) → **DIP** (ข้อใกล้ปลาย) → **TIP** (ปลายนิ้ว)

**กฎทอง:** พิกัดภาพ y = 0 ที่ขอบบน และเพิ่มลงล่าง ดังนั้นมือตั้งตรงฝ่ามือหันกล้อง → **นิ้วเหยียด (open) = TIP.y < PIP.y** · นิ้วงอ = TIP.y > PIP.y · แยกนิ้วกาง (V-sign) vs นิ้วชิด → ผลต่างของ x ระหว่างปลายนิ้วที่ติดกัน

```
def classifier(i_t,i_p, m_t,m_p, r_t,r_p, p_t,p_p):
    up = lambda t,p: t < p     # เหยียด
    f = [up(i_t,i_p), up(m_t,m_p),
         up(r_t,r_p), up(p_t,p_p)]
    if not any(f): return "ROCK"
    if all(f):     return "PAPER"
    if f[0] and f[1] and not f[2] and not f[3]:
        return "SCISSORS"
    return ""
```

**Game logic:** คอมสุ่ม `random.choice([...])` → เทียบกับผู้เล่น → You Win / You Lose / Tie → `render_template('game.html', …)` · **ROCK>SCISSORS>PAPER>ROCK** · เท่ากัน = Tie

## กับดักที่พลาดบ่อย

01 **High latency** ไม่ใช่คุณลักษณะ IoT — แต่ Scalability คือ (Enormous Scale)
02 Desktop PC ไม่ใช่ edge device · Jetson Nano เป็น
03 กล้อง Pi = MIPI CSI-2 · สตรีม = Flask + HTTP
04 kernel ใหญ่ขึ้น = เบลอแรงขึ้น · Laplacian = เหลือแต่ขอบ
05 Convolution พลิก kernel · filtering ไม่พลิก
06 CCTV ที่ถูก: on-device buffer สั้น → edge จำหน้า + กรอง event → cloud analytics + เก็บยาว
07 D2C = ไม่ผ่าน gateway — จับคำว่า **"directly"** ในโจทย์
08 ข้อดี Cloud-centric = พลังประมวลผล + เทรน ML ไม่ใช่ latency ต่ำ/privacy
09 ข้อเสีย Cloud OCR = network dependency + privacy
10 ขยะ OCR ทั่วภาพ → เพิ่ม text_threshold
11 Innovation = บริการ/มูลค่าใหม่ ไม่ใช่ if-else ไม่ใช่อัปไฟล์ขึ้น cloud
12 **"You Win!"** ขึ้นทันที = Notification ไม่ใช่ Visualization
13 MediaPipe = 21 จุด · นิ้วเหยียด TIP.y < PIP.y
14 UX metric = เวลาเรียนรู้การใช้แอป
15 โมเดลเก่าเจอข้อมูลใหม่แล้วแย่ = Maintainability
16 Explainability มีไว้ให้คนเชื่อถือผล ไม่ใช่ลด latency/storage

## ตัวเลข

| | |
|---|---|
| **3 วินาที** | |
| **21** | MediaPipe landmark/มือ |
| **32** | ข้อในข้อสอบเก่า · 1 คะแนน/ข้อ |
| **16% / 10%** | Lab รวม / Quiz → 25% |
| **11–25 fps** | ช่วงที่หน้าเริ่มเบลอ |
| **< 1 s** | latency ที่เกมยังเล่นได้ · 4 กล้อง |
| **0.5 s** | ไฟติดเมื่อจับความเคลื่อนไหว |
| **120/160 bpm** | threshold เตือน heart rate |
| **5 s** | กรอบเวลาส่ง alert สำเร็จ |
| **≥ 85%** | accuracy detection ที่ยอมรับ |
| **≥ 95%** | connectivity success · uptime 7 วัน |
| **±2%** | ความแม่นเซนเซอร์ความชื้นดิน |
| **12 ชม.** | แบต wearable |
| **2,000 / 8,000 ฿** | ต้นทุน farming / parking |